

Embedded Linux & System Software Interview Handbook

Chapter 4 - Board Bring-up (Part 6)

Topics: Memory Map, MMU, Exceptions, Kernel Panic Analysis and Real Debugging

4.26 Understanding the Memory Map

A memory map defines how the processor views the system address space. During board bring-up, engineers frequently inspect memory regions to verify that firmware, kernel and peripherals are placed correctly.

Region	Purpose
BootROM	First instructions after reset
Internal SRAM	BootROM/SPL execution
DDR	U-Boot, Kernel, RootFS
MMIO	Peripheral Registers
Flash (eMMC/NAND/NOR)	Persistent Storage

Physical vs Virtual Address

Boot loaders normally execute using physical addresses. After the Linux kernel enables the MMU, most kernel code uses virtual addresses. The MMU translates virtual addresses into physical addresses through page tables.

4.27 MMU (Memory Management Unit)

The MMU provides address translation, memory protection, process isolation and caching attributes. Without the MMU, modern operating systems cannot safely run multiple processes.

MMU Responsibilities

- Virtual-to-Physical address translation
- Memory protection
- Cache configuration
- Access permissions
- Process isolation

Interview Question

Why does Linux enable the MMU?

To provide virtual memory, isolate processes, protect kernel memory, and efficiently manage physical RAM.

4.28 CPU Exceptions

When software accesses invalid memory or executes an illegal instruction, the CPU raises an exception. Understanding exceptions is essential for debugging kernel crashes.

Exception	Typical Cause
-----------	---------------

Undefined Instruction	Invalid opcode
Data Abort	Invalid memory access
Prefetch Abort	Instruction fetch failure
Alignment Fault	Misaligned access
IRQ/FIQ	Hardware interrupt

4.29 Kernel Panic Analysis

A kernel panic indicates that Linux encountered a fatal error and cannot continue safely. Always read the first error message and the stack trace before making assumptions.

Typical Investigation Steps

1. Read the panic message.
2. Identify the failing function.
3. Examine register dump.
4. Review stack trace.
5. Check recently loaded drivers.
6. Compare with a known-good kernel.

Example

Kernel panic: Unable to mount root fs.

Possible causes include incorrect bootargs, missing storage driver, wrong Device Tree or filesystem corruption.

4.30 Real Debugging Scenario

Problem: Kernel crashes immediately after enabling a new Ethernet driver.

Approach:

- Capture complete UART log.
- Enable dynamic debug.
- Verify Device Tree compatible string.
- Check probe() return values.
- Validate mapped register addresses.
- Compare with previous working build.

How Senior Engineers Debug

Senior engineers avoid changing multiple things at once. They isolate one variable, reproduce the issue consistently, gather evidence from logs, JTAG and kernel traces, and only then implement a fix. This disciplined workflow significantly reduces debugging time.

Microsoft / NVIDIA Style Interview Questions

1. Explain virtual vs physical memory.
2. What is the purpose of the MMU?
3. How would you debug a kernel panic?
4. Explain Data Abort vs Prefetch Abort.
5. Why does Linux require page tables?
6. How would you debug a driver that crashes during probe()?
7. What information do you extract from a kernel stack trace?

Key Takeaways

Memory management and exception handling are core skills for platform engineers. Interviewers are interested in your debugging methodology as much as your technical knowledge. Always explain your reasoning, isolate the failing subsystem, and support conclusions with evidence from logs or hardware tools.